AN Code & Residue Code Pros + Cons

AN Code

Pros

- Single Widened ALU path
- Immune to zero-residue masking via single-bit flips adding an error of ± 2

Cons

- High decoding latency (via integer division)
- Complex multiplication overhead from $Ax * Ay = A^2xy$

Residue Code

Pros

- Zero Decoding Latency
- Parallel check logic operates concurrently with execution path

Cons

- Requires hardware duplcation (residue ALU & comparator logic)
- Vulnerable to zero-residue error masking during multiplication steps

Mitigations to Zero-Residue Error Masking in Residue Code

Original Question & My Context of Understanding

Question: *For multiplication, Residue Code can encounter the zero-residue problem. That is, using $C_N * C_M$ to check $N * M$, if $C_N = 0$, then any error that occurs in M cannot be detected. How to mitigate this problem? (25 pts, $C_N = N \bmod A$, $C_M = M \bmod A$)*

My understanding of the problem: *Given that N is a multiple of A, and that $C_N = 0$ plus $C_{N*M} = C_N * C_M$, this means that $0 = (0 * C_M) \bmod A$ which then expands to $0 = (0 * [M \bmod A]) \bmod A$. $\therefore$ if any errors occur to M, and therefore C_M, it would go completely undetected.*

Mitigation Strategies

- **Operand Offset Shifting N:** Whenever $C_N = 0$ is detected, turn N to $N' = N + 1$ and then compute $(N + 1) * M - M = N * M$ so that M (and C_M) can correctly occur, and then on the final product it generates it subtracts M to get the correct result.
- **Inverse Residue Coding:** If you detect that $C_N = 0$ simply compute the compliment so that you can detect any faults in M or C_M. Therefore you would use $C'_N = A - (N \bmod A)$ to replace C_N.